

Conferencia 4: listas

Programación · Ciencia de la Computación y Ciencia de Datos · MatCom · 2026-10-07

Con lo que sabes hasta ahora podrías leer las notas de un grupo y calcular el promedio: un acumulador que suma y un contador que cuenta. Pero no podrías calcular la mediana, ni imprimir las notas ordenadas, ni decir cuántas están por encima del promedio. El problema es el mismo en los tres casos: en cuanto lees la nota siguiente, la anterior se perdió, porque cada variable guarda un solo valor. Hoy aprendemos a guardar muchos valores con un solo nombre.

1. La lista

Una lista es una secuencia de valores. Se escribe entre corchetes, con los elementos separados por comas:

```
notas = [4, 5, 3, 5, 2, 4]
print(notas)
print(type(notas))
print(len(notas))
```

```
[4, 5, 3, 5, 2, 4]
<class 'list'>
6
```

`len` devuelve cuántos elementos tiene. Una lista puede estar vacía, y puede contener valores de cualquier tipo, incluso mezclados, aunque en la práctica casi siempre guardamos cosas del mismo tipo.

```
vacía = []
mezclada = [1, "dos", 3.0]
print(len(vacía), len(mezclada))
```

```
0 3
```

Los índices

A cada elemento se llega por su posición, que se llama **índice** y se escribe entre corchetes. **Los índices empiezan en cero.**

```
notas = [4, 5, 3, 5, 2, 4]
print(notas[0])
print(notas[1])
print(notas[5])
```

```
4
5
4
```

Que empiecen en cero tiene una consecuencia que hay que tener presente todo el tiempo: el último índice válido de una lista de n elementos es $n - 1$, no n . Pedir uno más allá del final es un error:

```
notas = [4, 5, 3, 5, 2, 4]
print(notas[6])
```

```
Traceback (most recent call last):
  File "<cell>", line 2, in <module>
    print(notas[6])
    ~~~~~^
IndexError: list index out of range
```

Ese `IndexError` lo vas a ver muchas veces este semestre, y casi siempre significa lo mismo: un ciclo que dio una vuelta de más.

Python admite además índices negativos, que cuentan desde el final. -1 es el último, -2 el penúltimo. Es cómodo y evita escribir `notas[len(notas) - 1]`:

```
notas = [4, 5, 3, 5, 2, 4]
print(notas[-1])
print(notas[-2])
```

```
4
2
```

2. Recorrer una lista

Aquí es donde el `for` de la clase dos enseña por fin su verdadera cara. Hasta ahora lo usábamos siempre sobre `range`, para contar. Pero `for` no sirve para contar: sirve para **recorrer una secuencia**, y `range` era solo una secuencia de números.

```
notas = [4, 5, 3, 5, 2, 4]

for nota in notas:
    print(nota, end=" ")
```

```
4 5 3 5 2 4
```

La variable `nota` toma en cada vuelta el valor de un elemento, en orden. No hay índices, no hay contador, no hay manera de pasarse del final. Esta es la forma normal de recorrer una lista en Python y debe ser tu primer instinto.

Con lo que ya sabes de acumuladores, calcular el promedio es inmediato:

```
notas = [4, 5, 3, 5, 2, 4]
suma = 0

for nota in notas:
    suma += nota

print(suma / len(notas))
```

```
3.8333333333333335
```

Cuando hace falta el índice

A veces necesitas saber en qué posición vas, por ejemplo para imprimir un número de orden. Entonces se recorre `range(len(...))`:

```
notas = [4, 5, 3, 5, 2, 4]

for i in range(len(notas)):
    print(f"Estudiante {i + 1}: {notas[i]}")
```

```
Estudiante 1: 4
Estudiante 2: 5
Estudiante 3: 3
Estudiante 4: 5
Estudiante 5: 2
Estudiante 6: 4
```

Y si necesitas las dos cosas, el índice y el elemento, Python tiene `enumerate`, que es más limpio que indexar a mano:

```
notas = [4, 5, 3, 5, 2, 4]

for i, nota in enumerate(notas):
    print(f"Estudiante {i + 1}: {nota}")
```

```
Estudiante 1: 4
Estudiante 2: 5
Estudiante 3: 3
Estudiante 4: 5
Estudiante 5: 2
Estudiante 6: 4
```

La regla: recorre con `for nota in notas` siempre que puedas, y usa el índice solo cuando de verdad lo necesitas.

Buscar

El operador `in` dice si un valor está en la lista, y `count` cuántas veces:

```
notas = [4, 5, 3, 5, 2, 4]
print(5 in notas)
print(6 in notas)
print(notas.count(5))
print(notas.index(3))
```

```
True
False
2
2
```

`index` devuelve la posición de la primera aparición, y da error si el valor no está.

3. Modificar una lista

Todo lo que hemos visto hasta hoy era **inmutable**: no se puede cambiar un número ni una parte de un texto. Cuando escribías `x = x + 1` no modificabas el 5, creabas un 6 y le ponías el mismo nombre.

Las listas son distintas. Una lista **se puede modificar en el lugar**, y eso cambia cómo hay que pensar sobre ellas.

```
notas = [4, 5, 3, 5, 2, 4]
notas[2] = 5
print(notas)
```

```
[4, 5, 5, 5, 2, 4]
```

Las operaciones que más se usan:

OPERACIÓN	QUÉ HACE
<code>lista.append(x)</code>	agrega x al final
<code>lista.insert(i, x)</code>	inserta x en la posición i
<code>lista.pop()</code>	saca y devuelve el último
<code>lista.pop(i)</code>	saca y devuelve el de la posición i
<code>lista.remove(x)</code>	elimina la primera aparición de x
<code>lista.sort()</code>	ordena la lista en el lugar
<code>lista.reverse()</code>	la invierte en el lugar

```
notas = [4, 5, 3]
notas.append(2)
print(notas)

ultima = notas.pop()
print(ultima, notas)

notas.insert(0, 5)
print(notas)
```

```
[4, 5, 3, 2]
2 [4, 5, 3]
[5, 4, 5, 3]
```

`append` es la que más vas a usar, porque es como se construye una lista a partir de una vacía:

```
cuadrados = []

for i in range(1, 6):
    cuadrados.append(i * i)
```

```
print(cuadrados)
```

```
[1, 4, 9, 16, 25]
```

La sorpresa más cara del semestre

Presta atención a esto, porque es la fuente de más errores de este curso que cualquier otra cosa. Con los números, asignar copia el valor:

```
a = 5
b = a
a = 6
print(a, b)
```

```
6 5
```

b sigue valiendo 5, como trazamos en la primera clase. Ahora lo mismo con listas:

```
a = [1, 2, 3]
b = a
a.append(4)
print(a)
print(b)
```

```
[1, 2, 3, 4]
[1, 2, 3, 4]
```

b cambió también, y nadie lo tocó. La razón es que `b = a` no copia la lista: copia el **nombre**. Después de esa línea hay una sola lista con dos nombres, y modificarla por cualquiera de los dos la modifica para ambos.

Esto no pasaba antes porque los números no se pueden modificar en el lugar; la única manera de cambiar a era asignarle otra cosa, y eso rompe el vínculo. Con una lista, `a.append(4)` no asigna nada: modifica el objeto que los dos nombres comparten.

Para copiar de verdad hay que pedirlo:

```
a = [1, 2, 3]
b = list(a)
a.append(4)
print(a)
print(b)
```

```
[1, 2, 3, 4]
[1, 2, 3]
```

Lo mismo pasa con los parámetros de una función. Si le pasas una lista a una función y la función la modifica, la lista de afuera queda modificada:

```
def agregar_cero(lista):
    lista.append(0)

notas = [4, 5, 3]
agregar_cero(notas)
print(notas)
```

```
[4, 5, 3, 0]
```

Compara con lo que viste la clase pasada, donde una función no podía cambiar el valor de un número de afuera. No es una contradicción: en los dos casos la función recibe el valor del argumento, pero el valor de una lista es la lista misma, no una copia de ella. A veces esto es exactamente lo que quieres. Cuando no lo es, la función debe trabajar sobre una copia y devolverla.

4. Rebanadas

Se puede pedir un pedazo de una lista con dos índices separados por dos puntos. El primero se incluye y el segundo no, la misma convención de range:

```
notas = [4, 5, 3, 5, 2, 4]
print(notas[1:4])
print(notas[:3])
print(notas[3:])
print(notas[::-1])
```

```
[5, 3, 5]
[4, 5, 3]
[5, 2, 4]
[4, 2, 5, 3, 5, 4]
```

Si omites el primer índice, empieza desde el principio; si omites el segundo, llega hasta el final. Un tercer número es el paso, y -1 recorre al revés, que es la forma más corta de invertir una lista.

Una rebanada siempre construye una **lista nueva**. Por eso `notas[:]` es otra manera de copiar, y por eso modificar la rebanada no toca el original:

```
notas = [4, 5, 3, 5, 2, 4]
copia = notas[:]
copia[0] = 100
print(notas)
print(copia)
```

```
[4, 5, 3, 5, 2, 4]
[100, 5, 3, 5, 2, 4]
```

5. El proyecto: las notas del grupo

Con esto ya podemos escribir el programa completo. Son tres piezas, cada una una función, como aprendimos la clase pasada.

Leer las notas hasta que el usuario escriba una línea en blanco:

```
def leer_notas():
    """Lee notas de la consola hasta una línea vacía. Devuelve la lista."""
    notas = []

    while True:
        linea = input("Nota (Enter para terminar): ")

        if linea == "":
            return notas

        notas.append(int(linea))
```

Calcular el promedio y el máximo. El máximo lo escribimos a mano, que es el patrón acumulador de la clase dos aplicado a una lista:

```
def promedio(notas):
    """Devuelve la media aritmética de una lista no vacía."""
    return sum(notas) / len(notas)

def maximo(notas):
    """Devuelve el mayor elemento de una lista no vacía."""
    mayor = notas[0]

    for nota in notas:
        if nota > mayor:
            mayor = nota

    return mayor

print(promedio([4, 5, 3, 5, 2, 4]))
print(maximo([4, 5, 3, 5, 2, 4]))
```

```
3.8333333333333335
5
```

Fíjate en que `maximo` empieza con `notas[0]` y no con cero. Empezar en cero funcionaría para notas, que son positivas, y fallaría con temperaturas bajo cero. Empezar por el primer elemento es siempre correcto, y a cambio la función exige que la lista no esté vacía.

Ahora que lo escribiste, te digo que Python ya lo trae: `max`, `min` y `sum` hacen esto mismo. Úsalos de aquí en adelante, pero el ejercicio valía la pena, porque en el examen te van a pedir el segundo mayor, o el mayor que cumple una condición, y ahí no hay función que lo haga por ti.

```
notas = [4, 5, 3, 5, 2, 4]
print(sum(notas), max(notas), min(notas), len(notas))
```

```
23 5 2 6
```

Lo que antes no se podía

Con las notas guardadas, aparecen las preguntas que la clase dos no podía responder. Cuántas están por encima del promedio, que exige recorrer la lista dos veces:

```
notas = [4, 5, 3, 5, 2, 4]
media = sum(notas) / len(notas)
sobre_la_media = 0
```

```
for nota in notas:
    if nota > media:
        sobre_la_media += 1

print(f"Promedio {media:.2f}, y {sobre_la_media} notas por encima")
```

```
Promedio 3.83, y 4 notas por encima
```

Y la mediana, que es el valor del medio una vez ordenadas:

```
def mediana(notas):
    """Devuelve la mediana de una lista no vacía."""
    ordenadas = sorted(notas)
    mitad = len(ordenadas) // 2

    if len(ordenadas) % 2 == 1:
        return ordenadas[mitad]

    return (ordenadas[mitad - 1] + ordenadas[mitad]) / 2

print(mediana([4, 5, 3, 5, 2, 4]))
print(mediana([4, 5, 3]))
```

```
4.0
4
```

sorted devuelve una lista nueva ordenada, y no toca la original; lista.sort() ordena en el lugar y no devuelve nada. Hoy los usamos como cajas negras. Dentro de tres semanas vamos a escribir nosotros el algoritmo que hay dentro.

6. Listas de listas

Un elemento de una lista puede ser cualquier cosa, incluso otra lista. Así se representa una tabla: las notas de varios estudiantes en varias asignaturas, una lista por estudiante.

```
tabla = [
    [5, 4, 5],
    [3, 3, 4],
    [2, 5, 3],
]

print(len(tabla))
print(tabla[0])
print(tabla[0][2])
```

```
3
[5, 4, 5]
5
```

tabla[0] es la primera fila, que es una lista. tabla[0][2] es la tercera nota de esa fila. El primer índice escoge la fila y el segundo la columna.

Para recorrerla hacen falta dos ciclos, uno dentro de otro:

```

tabla = [
    [5, 4, 5],
    [3, 3, 4],
    [2, 5, 3],
]

for fila in tabla:
    for nota in fila:
        print(nota, end=" ")
    print()

```

```

5 4 5
3 3 4
2 5 3

```

El `print()` sin argumentos al final del ciclo exterior es lo que salta de línea al terminar cada fila. Quítalo y verás las nueve notas en una sola línea.

Combinando lo de hoy, el promedio de cada estudiante sale en cuatro líneas:

```

tabla = [
    [5, 4, 5],
    [3, 3, 4],
    [2, 5, 3],
]

for i, fila in enumerate(tabla):
    print(f"Estudiante {i + 1}: promedio {sum(fila) / len(fila):.2f}")

```

```

Estudiante 1: promedio 4.67
Estudiante 2: promedio 3.33
Estudiante 3: promedio 3.33

```

Construir una matriz, y una trampa

Para hacer una tabla de ceros uno intenta lo obvio, y lo obvio está mal:

```

tabla = [[0] * 3] * 2
print(tabla)

tabla[0][0] = 9
print(tabla)

```

```

[[0, 0, 0], [0, 0, 0]]
[[9, 0, 0], [9, 0, 0]]

```

Cambiamos una casilla y se cambiaron dos. Es exactamente el problema de la sección 3: `[fila] * 2` no hace dos filas, hace **una sola fila con dos nombres**. La forma correcta es construir cada fila por separado:

```

tabla = []

for i in range(2):
    tabla.append([0] * 3)

```

```
tabla[0][0] = 9
print(tabla)
```

```
[[9, 0, 0], [0, 0, 0]]
```

[0] * 3 sí está bien, porque los ceros son inmutables y no importa que se compartan. Lo que no se puede repetir con * es una lista que después vas a modificar.

7. Resumen

- Una lista guarda muchos valores con un solo nombre. Se escribe con corchetes y su tamaño se pide con `len`.
- Los índices empiezan en cero y el último es `len - 1`. Los negativos cuentan desde el final. Pasarse da `IndexError`.
- `for nota in notas` recorre los elementos y es la forma normal. Usa el índice solo si lo necesitas, y entonces `enumerate`.
- Las listas son mutables: `append`, `pop`, `insert`, `remove`, `sort`.
- `b = a` no copia la lista, le pone otro nombre. Para copiar, `list(a)` o `a[:]`.
- Una función que recibe una lista puede modificarla, y el cambio se ve desde fuera.
- Una rebanada `a[i:j]` construye una lista nueva.
- `sum`, `max`, `min` y `sorted` existen, pero conviene saber escribirlos.
- Una lista de listas es una tabla: `tabla[filas][columnas]`, y se recorre con dos ciclos anidados.
- `[[0] * 3] * 2` no construye dos filas. Constrúyelas en un ciclo.

Ejercicios

1. Escribe `segundo_mayor(lista)` que devuelva el segundo elemento más grande, sin ordenar la lista y recorriéndola una sola vez. Piensa qué debe pasar con `[5, 5, 3]`.
2. Escribe `invertir(lista)` que devuelva una lista nueva con los elementos al revés, sin usar `[::-1]` ni `reverse`. Después escribe `invertir_en_el_lugar(lista)` que no devuelva nada y modifique la que recibió, intercambiando el primero con el último, el segundo con el penúltimo, y así. ¿Hasta dónde tiene que llegar el ciclo?
3. Escribe `sin_repetidos(lista)` que devuelva una lista nueva con los elementos de la original, en el mismo orden, pero sin repeticiones.
4. Escribe `es_capicua(lista)` que diga si la lista se lee igual al derecho y al revés. Resuélvelo sin construir ninguna lista nueva.
5. Con una tabla de números como la de la sección 6, escribe `transpuesta(tabla)` que devuelva una tabla nueva donde las filas son las columnas de la original. Comprueba que la transpuesta de la transpuesta es la tabla de partida.
6. Sin ejecutarlos, di qué imprime cada uno de estos tres programas. Después compruébalo.

(a)

```
a = [1, 2, 3]
b = a
c = a[:]
a[0] = 99
print(b[0], c[0])
```

(b)

```
def f(lista):  
    lista = [0, 0, 0]
```

```
def g(lista):  
    lista[0] = 0
```

```
x = [1, 2, 3]  
f(x)  
print(x)  
g(x)  
print(x)
```

(c)

```
numeros = [1, 2, 4, 3]  
  
for n in numeros:  
    if n % 2 == 0:  
        numeros.remove(n)  
  
print(numeros)
```

El tercero quiere eliminar los pares y no lo consigue. Explica por qué, y arréglalo. Después corre el mismo programa con [1, 2, 3, 4], donde sí da la respuesta correcta, y explica por qué esa entrada esconde el error. Es la clase de coincidencia que hace que un programa roto pase las pruebas.